JAVA COLLECTIONS • DATA STRUCTURES • INTERVIEW PREPARATION

HASHMAP
Professional Java Handbook

Core concepts • Built-in methods • Patterns • Dry runs • Interview problems

LEVEL	WHAT YOU WILL LEARN
Foundation	Key-value model, hashing, buckets, collisions, equals() and hashCode()
Practical Java	All commonly used HashMap methods with clear code examples
Problem solving	Frequency maps, lookup, grouping, prefix sums, sliding window patterns
Interview ready	Complexity, pitfalls, edge cases, practice set and solutions

A code-first reference for learning HashMap from beginner level to interview-level problem solving.

How to use this handbook

Read Chapters 1–5 in order, type the code yourself, and dry-run every example. Then study the patterns in Chapters 6–9 before attempting the practice set. Code examples use Java 8+ syntax unless noted.

Important: HashMap iteration order is not guaranteed. If output order matters, consider LinkedHashMap; if keys must remain sorted, consider TreeMap.

Contents

- 1. HashMap fundamentals and when to use it
- 2. How hashing works internally
- 3. Creating, inserting, reading and updating
- 4. Built-in methods reference
- 5. Iteration techniques
- 6. Frequency counting and common patterns
- 7. Interview problems with solutions
- 8. Complexity, pitfalls and debugging checklist
- 9. Practice questions and revision sheet

1. HashMap fundamentals

A HashMap stores data as key → value pairs. Each key is unique; values may repeat. It is part of java.util and implements the Map interface.

JAVA CODE

```
import java.util.HashMap;
import java.util.Map;

HashMap<String, Integer> scores = new HashMap<>();
scores.put("Aman", 90);
scores.put("Riya", 95);
scores.put("Aman", 98); // same key: old value is replaced

System.out.println(scores);          // order is not guaranteed
System.out.println(scores.get("Aman")); // 98
```

Generic types

In HashMap, K is the key type and V is the value type. For a frequency map, the array element is the key and its count is the value.

JAVA CODE

```
HashMap<Integer, Integer> frequency = new HashMap<>();
// key   = number from the array
// value = how many times that number appears
```

When should you use HashMap?

- Need fast average-time lookup by a key.
- Need to count occurrences of numbers, characters, or words.
- Need to remember a value associated with an identifier.
- Need to detect duplicates or find complements (for example, Two Sum).
- Need to group items by a common property.

Do not use HashMap when you need duplicate keys, guaranteed insertion order, or sorted-key order. Choose a suitable collection for those requirements.

2. How HashMap works internally

HashMap uses a key's hash code to help choose an internal bucket. The bucket then stores one or more entries. A simplified mental model is: key → hashCode() → bucket index → compare keys with equals().

JAVA CODE

```
// Conceptual model only (not the actual JDK implementation)
int hash = key.hashCode();
int bucketIndex = (capacity - 1) & hash;
// HashMap then checks keys in that bucket using equals().
```

Collision

A collision occurs when two different keys land in the same bucket. HashMap handles collisions internally; you normally do not manage buckets yourself.

equals() and hashCode() contract

• If two objects are equal according to equals(), they must return the same hashCode().

• Two unequal objects may still have the same hashCode(); that is a collision, not necessarily a bug.

• When creating custom key classes, implement equals() and hashCode() consistently.

• Do not mutate fields used by hashCode()/equals() while an object is being used as a key.

JAVA CODE

```
import java.util.Objects;

class Student {
    private final int id;
    Student(int id) { this.id = id; }

    @Override
    public boolean equals(Object obj) {
        if (this == obj) return true;
        if (!(obj instanceof Student)) return false;
        Student other = (Student) obj;
        return id == other.id;
    }

    @Override
    public int hashCode() {
        return Objects.hash(id);
    }
}
```

Performance: get/put/remove are typically O(1) average. In heavy-collision situations they can become slower; modern Java HashMap can treeify sufficiently large collision bins under qualifying conditions.

3. Creating, inserting, reading and updating

Create a HashMap

JAVA CODE

```
HashMap<Integer, String> map = new HashMap<>();
Map<Integer, String> map2 = new HashMap<>(); // interface reference
```

put(key, value)

Adds a new mapping or replaces the old value if the key already exists. Returns the previous value, or null if there was no previous mapping (with a caveat if null values are used).

JAVA CODE

```
map.put(101, "Asha");
map.put(102, "Kabir");
map.put(101, "Asha Sharma"); // updates key 101
```

get(key) and getOrDefault(key, defaultValue)

JAVA CODE

```
System.out.println(map.get(101));              // Asha Sharma
System.out.println(map.get(999));              // null
System.out.println(map.getOrDefault(999, "NA")); // NA
```

getOrDefault does not insert the default value. It only returns it when the key is absent.

containsKey(key) and containsValue(value)

JAVA CODE

```
if (map.containsKey(101)) {
    System.out.println("ID exists");
}
boolean hasName = map.containsValue("Kabir");
```

containsKey is generally much faster than containsValue. containsValue may scan the map.

remove(key) and remove(key, value)

JAVA CODE

```
map.remove(102);                 // remove mapping by key
map.remove(101, "wrong name");   // no removal: value does not match
map.remove(101, "Asha Sharma");  // removes only if both match
```

replace()

JAVA CODE

```
map.replace(101, "Asha Verma"); // only if key exists
map.replace(101, "Asha Verma", "Asha"); // conditional replacement
```

size(), isEmpty(), clear()

JAVA CODE

```
int count = map.size();
boolean empty = map.isEmpty();
map.clear(); // removes all mappings
```

4. Built-in HashMap methods: practical reference

Method	Purpose	Typical use
put(k, v)	Insert/update mapping	map.put(x, count)
putIfAbsent(k, v)	Put only if key is absent or mapped to null	Initialize a counter
get(k)	Read value; null if absent	Lookup
getOrDefault(k, d)	Read value or fallback	Frequency counting
containsKey(k)	Check key existence	Duplicate/lookup checks
containsValue(v)	Check if any value matches	Small maps / diagnostics
remove(k)	Remove mapping	Delete
remove(k, v)	Conditional removal	Safe conditional delete
replace(k, v)	Replace only existing key	Update
replace(k, old, new)	Replace only if old value matches	Conditional update
putAll(m)	Copy all mappings from another map	Merge maps
size()	Number of mappings	Count distinct keys
isEmpty()	Check whether map has no entries	Guard clauses
clear()	Remove all mappings	Reset map
keySet()	View of all keys	Key iteration
values()	View of all values	Value iteration

Method	Purpose	Typical use
entrySet()	View of key-value entries	Best for both key and value
forEach(action)	Visit each mapping	Compact traversal
computeIfAbsent(k, f)	Compute value if key absent/null	Grouping/list creation
computeIfPresent(k, f)	Compute value if key present and non-null	Update existing
compute(k, f)	Compute using old value (possibly null)	General update
merge(k, v, f)	Insert or combine with existing	Frequency aggregation

putIfAbsent

JAVA CODE

```
Map<String, Integer> visits = new HashMap<>();
visits.putIfAbsent("home", 0);
visits.putIfAbsent("home", 10); // remains 0
```

computeIfAbsent — very useful for grouping

JAVA CODE

```
Map<String, List<String>> groups = new HashMap<>();

groups.computeIfAbsent("fruit", k -> new ArrayList<>()).add("apple");
groups.computeIfAbsent("fruit", k -> new ArrayList<>()).add("mango");
groups.computeIfAbsent("drink", k -> new ArrayList<>()).add("tea");

// fruit -> [apple, mango]
// drink -> [tea]
```

compute / computeIfPresent

JAVA CODE

```
map.compute(101, (key, oldValue) ->
    oldValue == null ? "New" : oldValue + " Updated"
);

map.computeIfPresent(101, (key, oldValue) -> oldValue + "!");
```

merge — compact frequency counting

JAVA CODE

```
int[] nums = {2, 2, 3, 2, 3};
Map<Integer, Integer> freq = new HashMap<>();

for (int x : nums) {
    freq.merge(x, 1, Integer::sum);
}
// {2=3, 3=2} (display order may differ)
```

For beginners, getOrDefault + put is often easiest to understand. merge is a concise alternative.

5. Iterating through a HashMap

entrySet() — best when you need key and value

JAVA CODE

```
for (Map.Entry<Integer, Integer> entry : map.entrySet()) {
    int key = entry.getKey();
    int value = entry.getValue();
    System.out.println(key + " -> " + value);
}
```

keySet() — when you only need keys

JAVA CODE

```
for (Integer key : map.keySet()) {
    System.out.println(key);
```

```
    System.out.println(map.get(key));
}
```

values() — when you only need values

JAVA CODE

```
for (Integer value : map.values()) {
    System.out.println(value);
}
```

forEach()

JAVA CODE

```
map.forEach((key, value) ->
    System.out.println(key + " -> " + value)
);
```

Remember: entry.getKey() gives the key/element. entry.getValue() gives the mapped value/frequency. map.get(key) also retrieves the value directly.

6. Frequency counting: the most important DSA pattern

JAVA CODE

```
int[] arr = {1, 2, 2, 3, 1, 2};
Map<Integer, Integer> freq = new HashMap<>();

for (int x : arr) {
    freq.put(x, freq.getOrDefault(x, 0) + 1);
}

System.out.println(freq); // {1=2, 2=3, 3=1}
```

Dry run

Current number	Previous count	New count	Map after update
1	0 (default)	1	{1=1}
2	0 (default)	1	{1=1, 2=1}
2	1	2	{1=1, 2=2}
3	0 (default)	1	{1=1, 2=2, 3=1}
1	1	2	{1=2, 2=2, 3=1}
2	2	3	{1=2, 2=3, 3=1}

Print elements that occur exactly once

JAVA CODE

```
for (Map.Entry<Integer, Integer> entry : freq.entrySet()) {
    if (entry.getValue() == 1) {
        System.out.println(entry.getKey());
    }
}
```

Find the minimum frequency and its element(s)

JAVA CODE

```
int minFreq = Integer.MAX_VALUE;

for (int count : freq.values()) {
    minFreq = Math.min(minFreq, count);
}

for (Map.Entry<Integer, Integer> entry : freq.entrySet()) {
    if (entry.getValue() == minFreq) {
        System.out.println(entry.getKey() + " occurs " + minFreq + " time(s)");
```

```
    }
}
```

Do not return minFreq if the question asks for the single number. Return entry.getKey() when entry.getValue() == 1.

7. Interview problems with HashMap solutions

Problem 1 — Two Sum

Given an array and target, return indices of two numbers whose sum equals target. Store each previously seen number and its index.

JAVA CODE

```
class Solution {
    public int[] twoSum(int[] nums, int target) {
        Map<Integer, Integer> seen = new HashMap<>();

        for (int i = 0; i < nums.length; i++) {
            int need = target - nums[i];

            if (seen.containsKey(need)) {
                return new int[]{seen.get(need), i};
            }

            seen.put(nums[i], i);
        }
        return new int[]{-1, -1};
    }
}
```

Complexity: O(n) average time, O(n) space. Check before inserting the current value so one element is not used twice.

Problem 2 — Single Number (all others appear twice)

JAVA CODE

```
class Solution {
    public int singleNumber(int[] nums) {
        Map<Integer, Integer> freq = new HashMap<>();

        for (int x : nums) {
            freq.put(x, freq.getOrDefault(x, 0) + 1);
        }

        for (Map.Entry<Integer, Integer> e : freq.entrySet()) {
            if (e.getValue() == 1) {
                return e.getKey(); // key is the number, value is count
            }
        }
        return -1;
    }
}
```

Complexity: O(n) average time, O(n) space. Optimal XOR solution: O(n) time and O(1) extra space.

Problem 3 — First non-repeating character

JAVA CODE

```
class Solution {
    public int firstUniqChar(String s) {
        Map<Character, Integer> freq = new HashMap<>();

        for (char c : s.toCharArray()) {
            freq.put(c, freq.getOrDefault(c, 0) + 1);
        }

        for (int i = 0; i < s.length(); i++) {
            if (freq.get(s.charAt(i)) == 1) return i;
        }
        return -1;
    }
}
```

Two passes: first count; second scan the original string to preserve original order.

Problem 4 — Contains Duplicate

JAVA CODE

```
class Solution {
    public boolean containsDuplicate(int[] nums) {
        Map<Integer, Integer> freq = new HashMap<>();
        for (int x : nums) {
            if (freq.containsKey(x)) return true;
            freq.put(x, 1);
        }
        return false;
    }
}
```

A HashSet is even simpler for this task; HashMap is shown to practice key lookup.

Problem 5 — Majority Element (> n/2 times)

JAVA CODE

```
class Solution {
    public int majorityElement(int[] nums) {
        Map<Integer, Integer> freq = new HashMap<>();
        for (int x : nums) {
            int count = freq.getOrDefault(x, 0) + 1;
            freq.put(x, count);
            if (count > nums.length / 2) return x;
        }
        return -1;
    }
}
```

If the problem guarantees a majority element, the final -1 is only a fallback. Boyer–Moore voting achieves O(1) extra space.

Problem 6 — Intersection of Two Arrays II

JAVA CODE

```
class Solution {
    public int[] intersect(int[] a, int[] b) {
        Map<Integer, Integer> freq = new HashMap<>();
        List<Integer> ans = new ArrayList<>();

        for (int x : a) {
            freq.put(x, freq.getOrDefault(x, 0) + 1);
        }

        for (int x : b) {
            int count = freq.getOrDefault(x, 0);
            if (count > 0) {
                ans.add(x);
                freq.put(x, count - 1);
            }
        }

        int[] result = new int[ans.size()];
        for (int i = 0; i < ans.size(); i++) result[i] = ans.get(i);
        return result;
    }
}
```

The count is decreased after using a match, so duplicates are included only as many times as both arrays contain them.

Problem 7 — Isomorphic Strings

JAVA CODE

```
class Solution {
    public boolean isIsomorphic(String s, String t) {
        if (s.length() != t.length()) return false;

        Map<Character, Character> forward = new HashMap<>();
        Map<Character, Character> backward = new HashMap<>();
```

```
        for (int i = 0; i < s.length(); i++) {
            char a = s.charAt(i), b = t.charAt(i);

            if (forward.containsKey(a) && forward.get(a) != b) return false;
            if (backward.containsKey(b) && backward.get(b) != a) return false;

            forward.put(a, b);
            backward.put(b, a);
        }
        return true;
    }
}
```

Two maps enforce a one-to-one mapping in both directions.

Problem 8 — Group Anagrams

JAVA CODE

```
class Solution {
    public List<List<String>> groupAnagrams(String[] words) {
        Map<String, List<String>> groups = new HashMap<>();

        for (String word : words) {
            char[] chars = word.toCharArray();
            Arrays.sort(chars);
            String key = new String(chars);

            groups.computeIfAbsent(key, k -> new ArrayList<>()).add(word);
        }
        return new ArrayList<>(groups.values());
    }
}
```

Sorted letters form a canonical key. Time is approximately O(n · k log k), where k is maximum word length.

Problem 9 — Subarray Sum Equals K

Count continuous subarrays whose sum is k. Use prefix sums: if current prefix is sum, an earlier prefix sum - k creates a valid subarray.

JAVA CODE

```
class Solution {
    public int subarraySum(int[] nums, int k) {
        Map<Integer, Integer> prefixFreq = new HashMap<>();
        prefixFreq.put(0, 1);

        int sum = 0, answer = 0;

        for (int x : nums) {
            sum += x;
            answer += prefixFreq.getOrDefault(sum - k, 0);
            prefixFreq.put(sum, prefixFreq.getOrDefault(sum, 0) + 1);
        }
        return answer;
    }
}
```

Initialize prefixFreq with 0 → 1 to count subarrays starting at index 0. Works with negative numbers too. O(n) average time.

Problem 10 — Longest Consecutive Sequence

JAVA CODE

```
class Solution {
    public int longestConsecutive(int[] nums) {
        Map<Integer, Boolean> present = new HashMap<>();
        for (int x : nums) present.put(x, true);

        int best = 0;
        for (int x : present.keySet()) {
            if (!present.containsKey(x - 1)) {
                int current = x, length = 1;
                while (present.containsKey(current + 1)) {
                    current++;
```

```
                    length++;
                }
                best = Math.max(best, length);
            }
        }
        return best;
    }
}
```

A HashSet is the more natural collection here; this HashMap version demonstrates membership lookup. Expected O(n) time.

Problem 11 — Top K Frequent Elements

JAVA CODE

```
class Solution {
    public int[] topKFrequent(int[] nums, int k) {
        Map<Integer, Integer> freq = new HashMap<>();
        for (int x : nums) {
            freq.put(x, freq.getOrDefault(x, 0) + 1);
        }

        List<Integer> keys = new ArrayList<>(freq.keySet());
        keys.sort((a, b) -> freq.get(b) - freq.get(a));

        int[] ans = new int[k];
        for (int i = 0; i < k; i++) ans[i] = keys.get(i);
        return ans;
    }
}
```

This simple solution sorts distinct keys: O(m log m), where m is number of distinct values. For very large inputs, bucket sort or a heap may be preferable.

Problem 12 — Longest Substring Without Repeating Characters

JAVA CODE

```
class Solution {
    public int lengthOfLongestSubstring(String s) {
        Map<Character, Integer> lastSeen = new HashMap<>();
        int left = 0, best = 0;

        for (int right = 0; right < s.length(); right++) {
            char c = s.charAt(right);

            if (lastSeen.containsKey(c)) {
                left = Math.max(left, lastSeen.get(c) + 1);
            }

            lastSeen.put(c, right);
            best = Math.max(best, right - left + 1);
        }
        return best;
    }
}
```

HashMap stores the last index of each character. The window's left boundary never moves backwards. O(n) time.

8. More interview patterns

Pattern A — Value-to-index lookup

Store value → index when you need to quickly find where a previous value occurred. Common in Two Sum and index-distance problems.

Pattern B — Frequency map

Store item → count. Used in anagrams, majority element, duplicates, intersection, top-k frequency, and character counts.

Pattern C — Prefix sum + HashMap

Store prefixSum → number of times seen (or earliest index). Used for subarray sum equals k, equal 0/1 counts, and divisible-subarray problems.

Pattern D — Grouping

Store category/signature → list of items. Use computeIfAbsent to create a list only when the key first appears.

Pattern E — Last seen index

Store character/value → most recent index. Useful for sliding-window questions.

Pattern F — Two-way mapping

Use two maps to enforce bijection, as in Isomorphic Strings or word-pattern problems.

9. Complexity and collection choice

Operation / structure	Expected or typical cost	Notes
HashMap get / put / remove	O(1) average	Can degrade with collisions
containsKey	O(1) average	Hash-based lookup
containsValue	O(n)	May scan all values
Iterate entrySet / keySet / values	O(n)	Visits map contents
TreeMap get / put / remove	O(log n)	Keys sorted
LinkedHashMap get / put	O(1) average	Maintains encounter order
HashSet contains / add	O(1) average	Use when only membership matters

HashMap vs LinkedHashMap vs TreeMap

Collection	Order	Best when
HashMap	No guaranteed order	Fast key-value lookup
LinkedHashMap	Insertion/access order	Predictable iteration order or LRU-style designs
TreeMap	Sorted by key	Need sorted keys / range queries

10. Common mistakes and edge cases

• Returning entry.getValue() when the question asks for the element. Return entry.getKey().

• Assuming HashMap preserves insertion order. It does not guarantee that.

• Using == to compare String values. Use equals().

• Assuming get(key) returning null always means key is absent. A key can be mapped to null; use containsKey when this distinction matters.

• Modifying a map structurally while iterating with a for-each loop can cause ConcurrentModificationException. Use Iterator.remove() or suitable map methods.

• Using a mutable custom object as a key and then changing fields used in hashCode()/equals().

• Integer overflow in sums or counts for large constraints. Use long when required.

• Using containsValue for repeated lookups when a reverse map or set would be better.

• For output in original array/string order, do not rely on HashMap iteration; scan the original input after counting.

Safe removal while iterating

JAVA CODE

```
Iterator<Map.Entry<Integer, Integer>> it = map.entrySet().iterator();

while (it.hasNext()) {
    Map.Entry<Integer, Integer> entry = it.next();
    if (entry.getValue() == 0) {
        it.remove(); // safe through this iterator
    }
}
```

Null keys and values

HashMap permits one null key and multiple null values. Many other Map implementations have different null rules. Prefer avoiding null mappings unless the problem requires them.

11. Practice set — solve before reading solutions

For each problem, first identify what the map's key and value should represent.

#	Question	Suggested map design / pattern
1	Count frequency of each integer	number → count
2	Find the first repeated element	number → seen/count; preserve input scan
3	Find all elements appearing twice	number → count
4	Check whether two strings are anagrams	character → count
5	Find the most frequent character	character → count
6	Two Sum	number → earlier index
7	Count subarrays with sum K	prefix sum → frequency
8	Group words that are anagrams	sorted word → list
9	Find the longest substring with no repeats	character → last index
10	Find the intersection with duplicate counts	number → remaining count
11	Find the majority element	number → count
12	Find top K frequent values	number → count + heap/bucket
13	Find the first unique character index	character → count, then scan
14	Check isomorphic strings	two-way character mapping
15	Find pairs with a given difference	value → presence/count

12. Quick revision sheet

Question	Remember
What is a HashMap?	A key-value data structure; keys are unique.
Frequency map?	map.put(x, map.getOrDefault(x, 0) + 1)
Get key from entry?	entry.getKey()

Question	Remember
Get value from entry?	entry.getValue()
Get value by key?	map.get(key)
Check key?	map.containsKey(key)
Loop over pairs?	for (Map.Entry e : map.entrySet())
Count distinct keys?	map.size()
Default value inserted?	No; getOrDefault only returns a fallback.
Order guaranteed?	No. Use LinkedHashMap or TreeMap if needed.
Average lookup?	O(1) expected.
Custom object key?	Implement equals() and hashCode() consistently.

Final interview checklist

- Can you explain key, value, and why keys are unique?
- Can you write frequency counting without looking?
- Can you explain getOrDefault, putIfAbsent, merge, and computeIfAbsent?
- Can you iterate using entrySet and explain getKey vs getValue?
- Can you solve Two Sum, Group Anagrams, Subarray Sum K, and Longest Substring Without Repeating?
- Can you explain average O(1), collisions, equals(), and hashCode()?
- Can you identify when HashSet, LinkedHashMap, or TreeMap is a better choice?

Study advice: Do not memorize solutions line by line. Memorize the map design: what is the key, what is the value, when do you read it, and when do you update it.

End of handbook • Java HashMap • Beginner to interview preparation